

Following from [Applejak](#), Bulb is another teeny-tiny interpreted language that runs on SCHIP, taking up a good bit less than 4kb of RAM. While APLs provide many facilities for *list processing*, Bulb is, instead, a language built around the idea of manipulating *pairs*.

This language is *in principle* Turing-complete, but in practice severely limited by workspace RAM and the 16-level CHIP-8 call stack. The workspace consists of 256 atoms which may be numbers (unsigned bytes), symbols (interned, immutable strings), or pairs. Variables are stored in an environment structure which provides dynamic (read: cool and fun) scope for up to 64 simultaneous bindings, and is reduced slightly by nested function invocations.

Controls

CHIP-8 does not have an ordinary keyboard interface; just a hex keypad. Thus, to enter an expression at Bulb's [REPL](#), you use a menu-driven tree-builder. Cycle between choices with the up and down cursor keys (or W/S), and confirm a selection by pressing the right cursor key (or D). On mobile devices, swipe up, down, or right, respectively. Choices that are not yet "confirmed" will be drawn with an inverted background.

The [S-Expressions](#) comprising a program are entered as one of the following forms:

Symbol	Meaning
	Symbol. Symbols evaluate as their current dynamic binding, or nil if none exists. The symbols X,Y,Z,G have no meaning by default, and may be used freely for variables or argument names. The symbol F is initially bound to an example Llama expression, and may be redefined if desired.
	Number. Numbers are unsigned bytes 0-255 and evaluate as themselves.
	Definition. Build an S-Expression of the form (def SYMBOL EXPRESSION).
	Llama. Build an S-Expression of the form (Î» SYMBOL EXPRESSION).
	List. Build an S-Expression of the form (' (EXPRESSION...)), where the inner list may consist of any number of subexpressions. To terminate the list, choose ")" from the list of forms.
	Call. Build an S-Expression of the form (SYMBOL EXPRESSION...), where the list contains a number of subexpressions corresponding to the number of arguments the function bound to SYMBOL takes. (Its <i>valence</i> .)

Primitives

Bulb features a variety of built-in "primitive" routines, all of which have intuitively clear names. This should come as a welcome relief to anyone flustered by the use of esoteric symbols like "+" in source code.

Some primitives are *special forms*, which evaluate (or do not evaluate) their arguments according to unique whims. Special forms may be identified by their identical appearance to non-special forms.

Otherwise, each parenthesized term can be thought of as evaluating each argument in turn, before supplying them to the primitive or Llama indicated by the head of the term.

Name	Behavior
(cons x y)	Consecrate. Make a new pair from x and y.
(car x)	Automobile. Retrieve the first element from a pair.
(cdr x)	Compact Disk, Recordable. Retrieve the second element from a pair.
(not x)	Logical NOT. Returns a truthy value iff x is the number 0. Otherwise returns the number 0.
(atom x)	A Tom? Returns a truthy value iff x is a number or symbol. Otherwise returns the number 0.
(eq x y)	Exquisite. Returns a truthy value iff x and y are identical atoms. Pairs are <i>always</i> treated as non-identical.
(sum x y)	Addition. Returns the sum of the numbers x and y.
(diff x y)	Subtraction. Returns the difference of the numbers x and y.

- Lessen.
- (less x y) Returns a truthy value iff x feels intimidated by y.
6 is, for example, afraid of 7.
- (more x y) (A)more.
Returns a truthy value iff x is strictly larger than a big pizza y.
- It's Showtime!
- (show x) Print the value of x to the terminal output as a side-show (effect?),
and then return x unchanged.
- (iota x) Iota.
Build a list of numbers fom [0,n).
- Defecate.
- (def x y) Create or modify a binding in the local scope,
assigning y to the name given by the symbol x.
- Whereupon,
- (if x y z) It is the case that x is a truthy value, returns y, and otherwise returns z.
This is a *special form*- y and z are not eagerly evaluated.
- (' x) Quote.
A special form which returns x *without* first evaluating it.
- (() x) Nil.
If applied to anything, it returns Nil. May emit Hawking radiation as a side-effect.
- Llama.
- (Î» x y) A special form which produces an anonymous function with luxuriously soft fur,
taking an argument x and applying it to a body y.